Exercício: do Gherkin ao *Cucumber*

Quatro histórias de usuário, cenários escritos em linguagem de negócio e uma suíte de testes que roda de verdade — sobre o NewsFlow, a plataforma de newsletters e notificações da Vértice.

FORMATO	PRAZO	HISTÓRIAS	FERRAMENTA	ENTREGA
Duplas	3 dias	Até 4	Cucumber.js	Repositório Git

NESTE DOCUMENTO

01 O que você vai entregar 2
02 O sistema: NewsFlow 2
03 Histórias de usuário 3
04 Cenários em Gherkin 4
05 Exemplo resolvido 6
06 Automação com Cucumber 8
07 Checklist antes de entregar 9

CAPÍTULO 01

O que você vai entregar

> *Quatro histórias, os cenários que as comprovam e os testes que as executam.*

O trabalho tem três entregáveis encadeados. Cada um depende do anterior: um cenário só pode ser automatizado se estiver escrito, e só pode ser escrito se a história estiver clara.

#	ENTREGÁVEL	O QUE É
1	**Histórias de usuário**	No máximo **4** histórias no formato papel–objetivo–benefício, cada uma com critérios de aceitação em bullets.
2	**Cenários Gherkin**	Arquivos `.feature` com os cenários de cada história: caminho feliz, alternativas e exceções.
3	**Automação**	Projeto Cucumber.js com os steps implementados, suíte verde, relatório HTML e README.

Um trabalho está pronto quando alguém que nunca viu o sistema abre o repositório, lê os arquivos `.feature` e entende *o que* o sistema faz — sem abrir uma linha de código. Depois roda `npm test` e vê tudo passar. Se a pessoa precisa perguntar "o que esse cenário quer dizer?", a entrega ainda não está pronta, mesmo com os testes verdes.

Como a dupla se organiza

Uma dupla não comporta três cadeiras, mas comporta os três olhares. Dividam a responsabilidade principal e troquem de chapéu na revisão — é na revisão cruzada que os cenários ruins aparecem.

PAPEL NA DUPLA	RESPONSABILIDADE PRINCIPAL
Negócio	Dono(a) das histórias: recorte, prioridade, critérios de aceitação e a seção *Fora de escopo*.
Técnico	Dono(a) da automação e dos casos de borda: steps, World, execução e relatório.
Os dois	Revisam o trabalho um do outro: quem escreveu as histórias revisa os cenários, e vice-versa.

O cronograma de três dias

DIA	O QUE FAZER	PRONTO AO FIM DO DIA
1	Ler as regras, escolher as quatro histórias, escrever critérios de aceitação e a seção fora de escopo.	`US01` – `US04` em `docs/historias.md`.
2	Traduzir os critérios em cenários Gherkin e validar a sintaxe com `--dry-run`.	Arquivos `.feature` completos, com exceções e um Esquema do Cenário.
3	Implementar os steps sobre o projeto-base, deixar a suíte verde e gerar o relatório.	`npm test` passando, relatório HTML e README.

Se o tempo apertar, reduza a quantidade antes de reduzir a qualidade do texto: entregue as quatro histórias escritas e automatize pelo menos três por completo. Uma história com cenários bem escritos e sem automação vale mais que quatro cenários automatizados que ninguém entende.

CAPÍTULO 02

O sistema: NewsFlow

> *A plataforma que a Vértice usa para escrever newsletters e disparar notificações para 40 mil assinantes.*

A Vértice é uma empresa de tecnologia com assinantes espalhados pelo Brasil. Toda a comunicação escrita — o informativo semanal, os avisos de produto, os comunicados de manutenção — passa pelo NewsFlow. O redator escreve a edição, o revisor aprova, o gestor escolhe o segmento e agenda, e o sistema entrega por e-mail e por notificação no aplicativo. Por trás dessa rotina curta existem regras que se cruzam — e é justamente por isso que vocês terão de **escolher** quais quatro histórias contar.

ATOR	O QUE QUER
Redator(a)	Escrever e revisar a edição sem perder o que já fez, e ver como ela fica antes de enviar.
Revisor(a)	Conferir o conteúdo e liberar o envio sem virar gargalo.
Gestor(a)	Falar com o público certo, na hora certa, sem queimar a lista com excesso de envios.
Assinante	Receber o que interessa, no horário que não incomoda, e sair quando quiser.

Operação	Saber o que não foi entregue e por quê.

Regras de negócio

Toda história e todo cenário que vocês escreverem deve poder ser rastreado até uma destas regras.

ID	TEMA	REGRA
RN01	Rascunho	Toda edição nasce como rascunho e é salva automaticamente a cada **30 segundos**. Cada salvamento gera uma versão numerada, e o sistema mantém as **últimas 20 versões** para restauração.
RN02	Aprovação	Enviar ou agendar exige aprovação de um revisor. **O autor não aprova a própria edição.** Reprovação devolve a edição ao rascunho com um comentário obrigatório.
RN03	Agendamento	O envio é agendado para no mínimo **15 minutos** à frente, no fuso `America/Sao_Paulo`. Reagendar é permitido até **2 vezes**; cancelar, até **5 minutos** antes do horário marcado.
RN04	Segmentação	Uma edição vai para no máximo **5 segmentos**. Segmento com **zero assinantes** bloqueia o agendamento. Quem está em dois segmentos recebe **um único e-mail**.
RN05	Frequência	Um assinante recebe no máximo **2 newsletters por semana**. O excedente entra na fila da semana seguinte. E-mails transacionais (confirmação de cadastro, recibo) não contam para o limite.
RN06	Descadastro	Todo e-mail traz o link de descadastro, com efeito **imediato** e confirmação por e-mail. O retorno à lista exige **duplo opt-in**: link de confirmação válido por **48 horas**.
RN07	Falha de entrega	Até **3 tentativas**, com intervalos de **5, 15 e 45 minutos**. Depois da terceira, a entrega é marcada como falha definitiva. *Hard bounce* remove o assinante da base; *soft bounce* mantém.
RN08	Notificação	Notificações push respeitam a janela de silêncio das **22h às 8h**: o que for gerado nesse intervalo é entregue às 8h. Cada assinante recebe no máximo **3 notificações por dia**.

Vocabulário comum

Usem estes termos sempre com o mesmo significado — vocabulário inconsistente é a causa mais comum de cenário ambíguo.

TERMO	SIGNIFICADO NO NEWSFLOW
Edição	Uma newsletter: título, corpo, imagem de capa e data de envio. É o que se escreve, aprova e agenda.
Versão	Fotografia do conteúdo de uma edição em um instante do salvamento automático.
Segmento	Grupo de assinantes definido por critério (plano, cidade, interesse). Um assinante pode estar em vários.
Disparo	O envio de uma edição aprovada para os segmentos escolhidos.
Transacional	E-mail disparado por uma ação do assinante, não por campanha. Não conta no limite semanal.
Opt-in / opt-out	Consentimento para receber / pedido de saída da lista.

CAPÍTULO 03

Histórias de usuário

> *Uma história não é um requisito. É uma promessa de conversa com um valor explícito.*

A história responde a três perguntas em uma frase: **quem** quer, **o quê** e **por quê**. O "por quê" é o mais importante e o mais esquecido — sem ele, o time não consegue decidir o que é essencial e o que é "seria legal ter".

CAMPO	O QUE ESCREVER
ID	`US01` a `US04`. Serve para rastrear cenários, commits e bugs.
Título	Verbo no infinitivo + objeto: "Salvar rascunho", "Agendar envio".
História	Como *papel*, quero *objetivo*, para *benefício*.
Critérios	3 a 5 bullets com o que precisa ser verdade para a história estar pronta. Ainda em linguagem natural.
Regras cobertas	Quais RNs do Capítulo 02 a história atende. Toda história aponta para pelo menos uma.
Prioridade	Alta, média ou baixa, decidida por quem assume o papel de negócio, com justificativa de uma linha.

O que separa uma história boa de uma ruim

FRACA — E POR QUÊ	FORTE — E POR QUÊ

“Como usuário, quero uma tela de editor para escrever newsletter.” Fala de tela, não de valor. “Usuário” não é papel.	“Como redatora, quero ter meu texto salvo automaticamente enquanto escrevo, para não perder trabalho quando a conexão cair.” Papel real, resultado verificável, benefício concreto.
“Como desenvolvedor, quero criar a tabela `edicoes`.” É tarefa técnica. Não tem valor para ninguém de fora.	“Como revisor, quero ver as mudanças desde a versão aprovada, para não reler 40 mil caracteres a cada ajuste.” Quem pede é quem sente o problema.
“Como gestor, quero enviar a newsletter.” Não cabe em três dias: segmentação, agendamento, aprovação e frequência no mesmo saco.	“Como gestora, quero agendar o envio para um segmento específico, para que cada público receba só o que interessa.” Recorte vertical: uma regra, um comportamento, testável.

Como recortar as quatro

O erro clássico é recortar por camada técnica (“história do banco”, “história da API”, “história da tela”). Recorte por **fatia vertical**: cada história atravessa todas as camadas e entrega algo que o ator percebe sozinho. Teste rápido: se você removesse essa história do produto, algum ator sentiria falta de algo concreto? Se a resposta for “não”, é tarefa, não história.

Os critérios de aceitação são a ponte para a próxima etapa. Cada critério que descreve um comportamento observável vira um cenário; critérios que são apenas atributos (“a edição tem imagem de capa”) não viram cenário — viram pré-condição de outro. Não escrevam Gherkin direto: primeiro os bullets, depois a tradução.

MODELO PARA COPIAR EM DOCS/HISTORIAS.MD

```
## US01 — Salvar rascunho automaticamente

**Como** redatora
**Quero** que meu texto seja salvo enquanto escrevo
**Para** não perder trabalho quando a conexão cair

**Critérios de aceitação**
- O rascunho é salvo a cada 30 segundos de inatividade.
- Cada salvamento cria uma versão numerada.
- As últimas 20 versões podem ser restauradas.
- Restaurar uma versão não apaga as seguintes.

**Regras cobertas:** RN01
**Prioridade:** Alta — perda de conteúdo é a reclamação número um dos redatores.
**Cenários:** features/us01-rascunho.feature
```

O campo **Cenários** é o que fecha a rastreabilidade: dado um bug em produção, o time chega à história em dois cliques.

Quatro histórias, no máximo. Entregar oito histórias superficiais vale menos que quatro bem recortadas — a quinta é desconsiderada. Pelo menos duas regras ficarão de fora: elas vão para a seção *Fora de escopo* do README, cada uma com uma frase justificando a escolha.

CAPÍTULO 04

Cenários em Gherkin

> *Dado um contexto, quando algo acontece, então isto deve ser verdade.*

Gherkin é uma linguagem de palavras-chave, não de programação. Cada arquivo descreve uma *Funcionalidade* e contém *Cenários*. O Cucumber lê esse texto e procura, no código, um trecho correspondente a cada linha — é isso que transforma documentação em teste.

```
# language: pt
Funcionalidade: Agendar envio de newsletter
  Como gestora de comunicação
  Quero agendar o envio de uma edição aprovada
  Para que ela chegue no melhor horário sem eu precisar apertar o botão

  Contexto:
    Dado que estou autenticada como gestora
    E que existe a edição "Informativo #42" aprovada

  @feliz
  Cenário: Agendamento com antecedência suficiente
    Dado que o segmento "Clientes Premium" tem 1200 assinantes
    Quando agendo o envio para daqui a 30 minutos
    Então o disparo deve estar agendado
    E devo ver a mensagem "Envio agendado"
```

Palavras-chave que vocês vão usar

PALAVRA-CHAVE	PARA QUE SERVE
`Funcionalidade`	Agrupa os cenários de um comportamento. O texto “Como / Quero / Para” é só documentação — não é executado.
`Contexto`	Passos comuns a *todos* os cenários do arquivo. Roda antes de cada cenário, não uma vez só.
`Cenário`	Um exemplo concreto de comportamento. Deve poder ser lido isoladamente.
`Esquema do Cenário`	Modelo de cenário com variáveis `<assim>` preenchidas pela tabela de `Exemplos`.
`Dado / Quando / Então`	Contexto, ação e resultado. A ordem importa e não se repete dentro do cenário.
`E / Mas`	Continuação do passo anterior, do mesmo tipo. Três “E” seguidos costumam indicar cenário inflado.
`@tag`	Etiqueta para selecionar cenários na execução (`--tags @smoke`).

As cinco regras de escrita

1. **Escreva comportamento, não interface.** “Quando agendo o envio para daqui a 30 minutos” — não “quando clico no botão azul”. Se o botão mudar de cor, o teste não deveria quebrar.
2. **Um único Quando por cenário.** Se você precisa de dois “Quando”, você tem dois cenários.
3. **Cada cenário é independente.** Nenhum pode depender do que outro fez antes. O Cucumber executa em ordem imprevisível e em paralelo.
4. **Dados explícitos, nunca vagos.** “1200 assinantes”, não “muitos assinantes”. Se o valor varia, use Esquema do Cenário.
5. **Terceira pessoa, presente, sem jargão técnico.** Nada de “endpoint”, “JSON”, “tabela”, “mock” no texto do cenário.

Reescrevendo um cenário ruim

ANTES — ACOPLADO À SOLUÇÃO	DEPOIS — COMPORTAMENTO OBSERVÁVEL
Quando clico em `#btn-agendar` Então aparece um `div.toast` com texto	Quando agendo o envio para daqui a 30 minutos Então devo ver a mensagem “Envio agendado”
Dado que o banco tem 3 registros na tabela segmentos	Dado que o segmento “Clientes Premium” tem 1200 assinantes
Então o sistema faz um POST em /disparos e retorna 201	Então o disparo deve estar agendado para 10/03 às 9h00

Esquema do Cenário: onde estão os pontos do exercício

Sempre que uma regra tem **número de fronteira**, o esquema é a ferramenta certa. O agendamento (RN03) tem três: 15 minutos de antecedência mínima, 2 reagendamentos e 5 minutos para cancelar. Cada lado da fronteira produz um resultado diferente:

```
"k">Esquema do Cenário: Antecedência mínima para agendar
  "k">Dado que o segmento "Clientes Premium" tem 1200 assinantes
  "k">Quando agendo o envio para daqui a  minutos
  "k">Então devo ver a mensagem ""

  "k">Exemplos:
    | minutos | mensagem                            |
    | 60      | Envio agendado                      |
    | 16      | Envio agendado                      |
    | 15      | Envio agendado                      |
    | 14      | O envio precisa de ao menos 15 minutos |
    | 0       | O envio precisa de ao menos 15 minutos |
```

A tabela inclui o valor exatamente na fronteira e imediatamente ao lado (16 e 14). É aí que os bugs moram: um `>=` trocado por `>` passa despercebido em qualquer teste que só use “60 minutos”.

Com três dias de prazo, o mínimo por história é **um caminho feliz** + **um cenário de exceção**, e **um** Esquema do Cenário com pelo menos três linhas em todo o trabalho — na história com a fronteira numérica mais importante. Menos que isso é raso; seis cenários por história é sinal de que a história estava mal recortada.

CAPÍTULO 05

Exemplo resolvido

> *Um caso completo em outro domínio — biblioteca — para ver o encadeamento antes de fazer o seu.*

O exemplo não é do NewsFlow, justamente para não entregar o exercício pronto. Ele percorre as três etapas com o mesmo nível de detalhe esperado na entrega.

História	Como **estudante**, quero **renovar um empréstimo pela internet**, para **não precisar ir à biblioteca por causa de um prazo curto**.
Critérios	No máximo 2 renovações por empréstimo · não permitida quando há reserva na fila · nova devolução em 15 dias · confirmação por e-mail.

```
# language: pt
Funcionalidade: Renovação de empréstimo
  Como estudante
  Quero renovar um empréstimo pela internet
  Para não precisar ir à biblioteca por causa de um prazo curto

  Contexto:
    Dado que estou autenticado como estudante
    E tenho o livro "Engenharia de Software" emprestado

  @feliz
  Cenário: Renovação dentro do prazo e sem reserva
    Dado que o empréstimo vence em 5 dias
    E não há reserva para este livro
    Quando solicito a renovação
    Então a nova data de devolução deve ser 15 dias a partir de hoje
    E devo ver a mensagem "Empréstimo renovado com sucesso"

  @excecao
  Cenário: Renovação bloqueada por reserva na fila
    Dado que o empréstimo vence em 5 dias
    E outro usuário reservou este livro
    Quando solicito a renovação
    Então devo ver a mensagem "Este livro tem reserva na fila"
    E a data de devolução não deve ser alterada

  @limite
  Esquema do Cenário: Limite de duas renovações
    Dado que o empréstimo já foi renovado  vez(es)
    Quando solicito a renovação
    Então devo ver a mensagem ""

    Exemplos:
      | vezes | mensagem                       |
      | 0     | Empréstimo renovado com sucesso |
      | 1     | Empréstimo renovado com sucesso |
      | 2     | Limite de renovações atingido   |
```

Abaixo, os steps que fazem esse texto executar. Cada linha do Gherkin encontra uma expressão; o texto entre chaves é capturado como argumento — `{string}` pega texto entre aspas, `{int}` pega inteiro.

```
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')

Given('que o empréstimo já foi renovado {int} vez(es)', function (vezes) {
  this.emprestimo.renovacoes = vezes
})

Given('não há reserva para este livro', function () {
  this.livro.reservas = 0
})

When('solicito a renovação', function () {
  this.resultado = this.servico.renovar(this.emprestimo)
})

Then('devo ver a mensagem {string}', function (esperada) {
  assert.equal(this.resultado.mensagem, esperada)
})

Then('a nova data de devolução deve ser {int} dias a partir de hoje', function (dias) {
  const esperada = new Date()
  esperada.setDate(esperada.getDate() + dias)
  assert.equal(this.emprestimo.vencimento.toDateString(), esperada.toDateString())
})
```

O que observar: o `Contexto` carrega só o que todos os cenários compartilham; o cenário de exceção muda *uma* condição e verifica *outra* consequência; o esquema cobre as três posições possíveis do contador (0, 1, 2) sem repetir cenário; nenhum passo fala de tela ou banco de dados — e a verificação de data compara datas, não strings formatadas.

CAPÍTULO 06

Automação com Cucumber

> *O Gherkin descreve. O Cucumber executa. Os steps fazem a ponte.*

O **Cucumber** lê os arquivos `.feature`, encontra o código correspondente a cada passo e executa. Existe para Java, Ruby, Python e outras linguagens; usamos **Cucumber.js** com Node.js porque a instalação é de um comando e não exige configurar projeto nem servidor. A convenção de pastas abaixo evita configuração extra: o Cucumber procura `.feature` em `features/` e código em `features/step_definitions/` e `features/support/`.

```
mkdir newsflow-bdd && cd newsflow-bdd
npm init -y
npm install --save-dev @cucumber/cucumber
```

```
newsflow-bdd/
├── cucumber.js
├── docs/historias.md
├── features/
│   ├── us01-rascunho.feature
│   ├── support/world.js
│   └── step_definitions/rascunho.steps.js
└── relatorios/cucumber-report.html
```

A configuração e o atalho de execução ficam assim:

```
// cucumber.js
module.exports = {
  default: {
    require: ['features/support/**/*.js', 'features/step_definitions/**/*.js'],
    format: ['progress', 'html:relatorios/cucumber-report.html'],
    publish: false
  }
}

// package.json
"scripts": { "test": "cucumber-js", "test:dry": "cucumber-js --dry-run" }
```

Rode `npx cucumber-js` e a ferramenta lista os passos sem implementação, já com o esqueleto pronto para colar. Implemente um por um:

```
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')

Given('que existe a edição {string} com o texto {string}', function (titulo, texto) {
  this.edicao = this.servico.criarEdicao(titulo, texto)
})

When('eu espero {int} segundos sem digitar', function (segundos) {
  this.avancarSegundos(segundos)
})

Then('o rascunho deve ter {int} versões salvas', function (quantidade) {
  assert.equal(this.edicao.versoes.length, quantidade)
})
```

O World: memória de um cenário

O `this` dentro dos steps é o *World*. Ele é recriado antes de cada cenário, o que garante a independência do capítulo anterior — se você guardar estado em variável global, um cenário passa a depender do outro e a suíte fica frágil.

```
// features/support/world.js
const { setWorldConstructor } = require('@cucumber/cucumber')
const Servico = require('./dominio')

class MundoDoNewsFlow {
  constructor() {
    this.servico = new Servico()
    this.edicao = null
    this.resultado = null
  }

  avancarSegundos(segundos) {
    this.servico.avancarRelogio(segundos * 1000)
  }
}
setWorldConstructor(MundoDoNewsFlow)
```

Rodar e ler o resultado

COMANDO	O QUE FAZ
`npx cucumber-js`	Executa todos os `.feature` e mostra quantos cenários e passos passaram.
`npx cucumber-js --dry-run`	Valida a sintaxe e confere se todo passo tem implementação, sem executar. Use antes de cada commit.
`npx cucumber-js --tags "@smoke"`	Roda só os cenários marcados. Combina: `--tags "@smoke and not @wip"`.
`npx cucumber-js --parallel 2`	Executa em paralelo — só funciona se os cenários forem independentes.
`npm run test:dry`	O atalho documentado no README. É o comando que comprova a entrega.

Uma execução saudável termina assim, com todos os cenários passando e nenhum passo pendente:

```
......................
6 scenarios (6 passed)
21 steps (21 passed)
0m00.021s (executing steps: 0m00.002s)
```

Relatório e evolução

A configuração acima já gera `relatorios/cucumber-report.html`: um relatório navegável com cada cenário, cada passo e o tempo de execução — é o artefato que vocês anexam. Neste exercício o domínio pode ser em memória; o foco é o encadeamento Gherkin → step → verificação. Em um projeto real, o corpo do step chama o sistema de verdade: por API (`await fetch('/api/disparos', …)`), que é rápido e estável e serve para a maioria dos cenários; ou pela interface, com Playwright, reservado a poucos casos, porque é lento e quebra a cada mudança de layout.

Usar o Cucumber como ferramenta de teste de interface para tudo produz centenas de cenários lentos, frágeis e que ninguém de negócio consegue ler. A proporção que funciona: **muitos cenários na camada de domínio ou API, poucos na interface** — e todos escritos em linguagem de negócio.

CAPÍTULO 07

Checklist antes de entregar

> *Passe item por item antes de enviar o repositório.*

Histórias

- [ ] No máximo 4 histórias, numeradas `US01`–`US04`.
- [ ] Papel, objetivo e benefício reais — o benefício não é “para o sistema funcionar”.
- [ ] De 3 a 5 critérios de aceitação verificáveis.
- [ ] Cada história aponta para pelo menos uma RN.
- [ ] README com a seção *Fora de escopo* justificada.

Cenários

- [ ] Cada história tem um cenário de sucesso e um de exceção.
- [ ] Pelo menos um Esquema do Cenário com três ou mais linhas, cobrindo fronteiras.
- [ ] Nenhum cenário depende do resultado de outro.
- [ ] Nenhum passo cita elemento de tela, endpoint, tabela ou biblioteca.
- [ ] Um único `Quando` por cenário.
- [ ] Os números aparecem no texto (15 minutos, 2 reagendamentos), não “prazo adequado”.
- [ ] Todos os arquivos começam com `# language: pt`.

Automação

- [ ] `npm install && npm test` funciona em máquina limpa.
- [ ] `--dry-run` termina sem passos pendentes.
- [ ] Suíte verde — nada comentado para “passar”.
- [ ] Steps reutilizáveis: o mesmo passo serve a mais de um cenário.
- [ ] Nenhum `sleep` fixo.
- [ ] Estado no World, não em variável global.
- [ ] Relatório HTML gerado e no repositório.
- [ ] README explica como rodar em até cinco linhas.

Como entregar

- [ ] Repositório Git com o projeto, sem `node_modules`.
- [ ] `docs/historias.md` com as quatro histórias.
- [ ] `.feature` em `features/`, um por história.
- [ ] Saída de `npm test` colada no README.